

TRAIN TEST SPLIT

1. `sklearn.model_selection.train_test_split(`
 `* array,`
 `test_size = None,`
 `train_size = None,`
 `random_state = None,`
 `shuffle = True,`
 `stratify = None)` → $y/n \rightarrow$ Ensures fair distr of data for training / testing

2. `import pandas as pd`
 `df = pd.read_csv('path.csv')`
 `df.head()`
 `df.columns` → lets say 10 columns.
 `y = df['diabetes']` ← `X = df.drop(['diabetes', axis=1])`
 from `sklearn.model_selection` import `train_test_sp...`

`X_train, X_test, y_train, y_test = train_test_split(`
 `X, y, train_size = 0.7, stratify = y,`
 `random_state = 2529)`

`X_train, X_test, y_train, y_test.shape`

Ques	Ans
0	0
0	0
0	0
0	0
0	0
0	0
0	0

X y

Train

Test

- This is like setting fixed Seed
- It ensures that everytime you run code, you get same split.

👉 Complete Workflow Summary

1. Split data → `train_test_split`
2. Choose model → KNN/Logistic/etc
3. Train model → `model.fit`
4. Predict → `model.predict`
5. Evaluate → accuracy, confusion matrix
6. Improve → change parameters

This is the standard ML pipeline.

LOGISTIC REGRESSION

```
from sklearn.linear_model import LogisticRegression
```

Creating Model

```
model = LogisticRegression()
```

Train (fit) the Model

```
model.fit(X_train, y_train)
```

We give both Dues (X) and Dues (y) for kg

Predicting test data.

```
y_pred = model.predict(X_test)
```

Evaluating the Model

```
from sklearn.metrics import accuracy_score,  
confusion_matrix  
print(accuracy_score(y_test, y_pred)  
print(confusion_matrix(y_test, y_pred))
```

Predicting Probability

```
model.predict_proba(X_test)
```